

# 중간고사 평가보고서

심해 탐험대 게임 제작 실기시험

## Unity를 이용한 2D 게임프로그래밍

동명대학교 게임공학과 | 강영민

2026년 1학기

### 문서 개요

|       |                                   |
|-------|-----------------------------------|
| 시험일   | 2026년 4월 21일 (월)                  |
| 응시 인원 | 27명 (제출 25명, 미제출 2명)              |
| 시험 시간 | 120분                              |
| 총점    | 100점 (4과제 16세부항목)                 |
| 평가 방법 | 자가평가 + 코드 분석(연동 검증)               |
| 코드 표본 | 25명 분 약 250개 .cs 파일, 추정 12,000여 줄 |
| 평균 점수 | 29.3점 (중앙값 35점, 표준편차 약 18점)       |
| 최고점   | 78점 (학생20)                        |

## Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>시험 개요</b>                                       | <b>4</b>  |
| 1.1      | 시험 설계의 기본 방향                                       | 4         |
| 1.2      | 과제 구성과 배점                                          | 4         |
| 1.3      | 학습 목표와 평가 항목의 정렬                                   | 5         |
| <b>2</b> | <b>평가 방법론</b>                                      | <b>6</b>  |
| 2.1      | 왜 코드만으로는 평가가 부족한가                                  | 6         |
| 2.2      | 3단계 판정 체계                                          | 6         |
| 2.3      | 자가평가 항목과 시험과제 매핑                                   | 6         |
| 2.4      | 코드 분석 절차                                           | 6         |
| 2.5      | 개념 구현 30% 반영의 근거                                   | 7         |
| 2.6      | 본 평가의 한계                                           | 7         |
| <b>3</b> | <b>자가평가 결과</b>                                     | <b>8</b>  |
| 3.1      | 자가평가 원시 데이터                                        | 8         |
| 3.2      | 자가평가 항목별 동작 보고 분포                                  | 9         |
| <b>4</b> | <b>코드 분석: 패턴별 상세 비교</b>                            | <b>10</b> |
| 4.1      | 패턴 1: 입력 처리 – 신형 Input System vs 구식 Input API      | 10        |
| 4.2      | 패턴 2: 충돌 처리 – 컴포넌트 분기의 3가지 길                       | 11        |
| 4.3      | 패턴 3: 데이터 자산 분리 – ScriptableObject                 | 13        |
| 4.4      | 패턴 4: Wave/Follower 연동 – Spawner→Follower→Waypoint | 14        |
| 4.5      | 패턴 5: 발사 메커니즘 – Coroutine vs InvokeRepeating       | 15        |
| 4.6      | 패턴 6: 점수 표시 – TextMeshProUGUI vs Debug.Log         | 16        |
| 4.7      | 패턴 7: 카메라 흔들림 – Singleton vs 인스펙터 참조 vs 미연결        | 17        |
| 4.8      | 연동 체인 검증 요약                                        | 18        |
| 4.9      | 코드 품질 vs 자가평가 매트릭스                                 | 19        |
| <b>5</b> | <b>최종 평가 결과</b>                                    | <b>20</b> |
| 5.1      | 16개 항목별 최종 점수                                      | 20        |
| 5.2      | 과제별 소계 및 통계                                        | 23        |
| <b>6</b> | <b>점수 분포 분석</b>                                    | <b>24</b> |
| 6.1      | 총점 히스토그램                                           | 24        |
| 6.2      | 등급 분포                                              | 24        |
| 6.3      | 자가평가 점수 vs 최종 점수 산점도                               | 25        |
| <b>7</b> | <b>개별 답안 사례 분석</b>                                 | <b>26</b> |
| 7.1      | 학생20 (78점, A등급) – 모범 답안                            | 26        |
| 7.2      | 학생08 (52점, B등급) – 표준 준수 + 일부 미완                    | 27        |
| 7.3      | 학생04 (50점, B등급) – 과제2 강제, 과제3 단절                   | 28        |
| 7.4      | 학생12 (47점, B등급 경계) – 다층 분리 시도                      | 29        |
| 7.5      | 학생01 (46점, C등급) – 코드 최상/실행 미완 (음영 강조 사례)           | 29        |
| 7.6      | 학생09 (46점, C등급) – 모던 C# 문법 (음영 강조 사례)              | 30        |
| 7.7      | 학생15 (35점, D-C 경계) – 코드/실행 극단적 분리 (음영 강조 사례)       | 31        |
| 7.8      | 학생23 (37점, C등급) – 동작하는 게임의 낮은 점수                   | 32        |
| 7.9      | 사례 비교 매트릭스                                         | 35        |

|     |                           |    |
|-----|---------------------------|----|
| 8   | 종합 소견                     | 36 |
| 8.1 | 과제별 완성도 패턴 . . . . .      | 36 |
| 8.2 | 4가지 “불일치 패턴” 종합 . . . . . | 36 |
| 8.3 | 5가지 주요 시사점 . . . . .      | 36 |
| 8.4 | 향후 수업 방향 . . . . .        | 37 |

## 1 시험 개요

### 1.1 시험 설계의 기본 방향

본 중간고사는 “심해 탐험대 (Deep Sea Explorer)”라는 가상의 2D 슈팅 게임을 Unity로 처음부터 제작하는 실기 형태로 출제되었다. 단순 코드 암기로 해결할 수 없도록 **컴포넌트 기반 설계**, **ScriptableObject 기반 데이터 분리**, **코루틴 기반 비동기 흐름**이라는 세 가지 핵심 기법을 모두 요구하는 4개 과제로 구성하였다.

- **인터넷·AI 도구 사용 허용** – 단순 “코드 받아쓰기”로는 부분 점수만 가능하도록 다중 클래스 간 **연동(integration)**을 핵심 평가 항목으로 설계.
- **컴포넌트 기반 설계** – GetComponent<T>() 패턴으로 적/생물을 판별하고, Health와 DamageDealer의 역할 분리를 평가.
- **데이터 자산 분리** – 웨이브 설정을 ScriptableObject로 빼내어 **디자이너 작업과 프로그래머 작업의 경계**를 체험하도록 설계.
- **제출물**: Unity 프로젝트의 .cs 파일만 압축 제출. Inspector 연결, 프리팹 구성, 레이어 설정 등 에디터 작업은 학생 자가평가로 보고.

### 1.2 과제 구성과 배점

각 과제는 **독립적으로 채점 가능**하지만, 과제 3(전투 시스템)은 과제 1(잠수함)과 과제 2(해류 생물)의 산출물을 **조립**하는 단계로 설계되어 있어, 앞 과제의 미완성은 뒤 과제로 **연쇄 단절**된다.

| 과제 | 주제        | 세부 항목 (배점)               | 소계  |
|----|-----------|--------------------------|-----|
| 1  | 잠수함 조종    | 1-1 오브젝트 구성 (7)          | 25  |
|    |           | 1-2 조종 스크립트 구현 (12)      |     |
|    |           | 1-3 경계 제한 (6)            |     |
| 2  | 해류 생물 시스템 | 2-1 SeaWaveConfigSO (6)  | 25  |
|    |           | 2-2 프리팹 구성 (5)           |     |
|    |           | 2-3 해류 경로/Wave (4)       |     |
|    |           | 2-4 Spawner/Follower (5) |     |
|    |           | 2-5 난이도 설계 (5)           |     |
| 3  | 전투 시스템    | 3-1 Launcher (9)         | 35  |
|    |           | 3-2 레이어 설정 (5)           |     |
|    |           | 3-3 Health (5)           |     |
|    |           | 3-4 컴포넌트 분기 (8)          |     |
|    |           | 3-5 DamageDealer (3)     |     |
|    |           | 3-6 ScoreKeeper (5)      |     |
| 4  | 연출 효과     | 4-1 자유 이펙트 (9)           | 15  |
|    |           | 4-2 수중 충격파 (6)           |     |
| 합계 |           |                          | 100 |

### 1.3 학습 목표와 평가 항목의 정렬

배점은 **설계 능력 단계**에 따라 가중되어 있다.

| 학습 단계         | 배점 비중 | 대표 평가 항목                                                            |
|---------------|-------|---------------------------------------------------------------------|
| 1. 단일 스크립트 작성 | 25%   | 1-1, 1-2, 1-3 (잠수함 조종)                                              |
| 2. 데이터 자산 설계  | 11%   | 2-1 SeaWaveConfigSO                                                 |
| 3. 다수 스크립트 연동 | 49%   | 2-4, 3-1~3-6 (Spawner→Follower, Launcher→DamageDealer→Health→Score) |
| 4. 시각·체험 폴리시  | 15%   | 4-1, 4-2 (이펙트, 카메라 흔들림)                                             |

**관찰:** 49%가 “다수 스크립트 연동”에 배정되어 있다는 사실은, 이 시험이 개별 클래스의 정확성이 아니라 객체 간 메시지 흐름의 완결성을 평가하려는 의도임을 명확히 한다. 후술하는 “5개 연동 체인” 분석은 이 의도를 검증하기 위한 도구이다.

## 2 평가 방법론

### 2.1 왜 코드만으로는 평가가 부족한가

Unity 게임은 **코드**와 **에디터 작업**의 결합으로 동작한다. 대표적 에디터 작업은 다음과 같다.

- Inspector에서 [SerializeField] 필드에 프리팹·컴포넌트 할당
- Project 창에서 [CreateAssetMenu] ScriptableObject 자산 생성
- Hierarchy에서 GameObject 구성, 부모-자식 관계, Tag 설정
- Edit 메뉴의 Project Settings에서 Layer 추가, Physics2D 충돌 매트릭스 조정
- Input Actions 에셋에 “Move”, “Fire” 액션 등록 및 키 바인딩

코드가 완벽해도 위 중 하나라도 누락되면 **NullReferenceException** 또는 **무반응**으로 게임이 동작하지 않는다. 본 시험은 코드만 제출 받았으므로, 에디터 작업 완료 여부는 학생의 자가 보고에 의존할 수밖에 없다. 이를 위해 도입한 것이 **3단계 판정 체계**이다.

### 2.2 3단계 판정 체계

| 판정 기준과 반영률 |    |                       |      |
|------------|----|-----------------------|------|
| 판정         | 기호 | 조건                    | 반영률  |
| 실제 동작      | ◎  | 자가평가=1 AND 코드가 정상 구현됨 | 100% |
| 개념 구현      | ○  | 자가평가=0 AND 코드가 존재함    | 30%  |
| 구현 미비      | ×  | 코드 없음 또는 빈 파일         | 0%   |

### 2.3 자가평가 항목과 시험과제 매핑

학생들은 시험 종료 직후 8개 항목에 대해 동작 여부(1/0)를 자가 보고하였다. 채점은 자가평가의 “1” 보고를 **Inspector 연결까지 완료된 상태로** 간주하고, “0” 보고는 **코드는 있으나 에디터 연결 미완료**로 해석한다.

| 번호 | 자가평가 항목 | 대응 시험 과제                           | 배점 |
|----|---------|------------------------------------|----|
| 1  | 프로젝트 생성 | 1-1 오브젝트 구성                        | 7  |
| 2  | 입력처리    | 1-2 조종스크립트 + 1-3 경계제한              | 18 |
| 3  | 적출현     | 2-1~2-5 해류생물 전체                    | 25 |
| 4  | 게임처리    | 3-1~3-4 전투 핵심                      | 27 |
| 5  | 게임상황표시  | 3-5 DamageDealer + 3-6 ScoreKeeper | 8  |
| 6  | 캐릭터폭발   | 4-1 자유이펙트                          | 9  |
| 7  | 카메라진동   | 4-2 수중충격파                          | 6  |
| 8  | 특수이펙트   | (보너스, 기본 100점에 불포함)                | 0  |

### 2.4 코드 분석 절차

1. **개별 스크립트 정적 검토**: 각 .cs 파일의 클래스 구조, 필드 가시성, 메서드 시그니처, 사용된 Unity API 종류 (신형 Input System vs 구식 Input.GetAxis)를 확인.

2. **연동 체인 검증**: 5개 핵심 체인의 “호출 그래프”를 수동으로 추적. 메서드 호출이 한 군데라도 끊어지면 “단절”로 판정.
- **Chain 1**: Input → Movement  
OnMove(InputValue) → rawInput 저장 → Update() → transform.position 갱신 → Mathf.Clamp 경계 처리
  - **Chain 2**: Spawner → Follower → Wave  
Instantiate(prefab) → follower.SetWaveConfig(so) → Update() → Vector3.MoveTowards(waypoint)
  - **Chain 3**: Attack → Damage → Health  
Launcher.FireContinuously() → 발사체에 DamageDealer 부착 → 충돌 시 Health.OnTriggerEnter2D() 에서 GetComponent<DamageDealer>() → TakeDamage(d.GetDamage())
  - **Chain 4**: Component Branch → Score/Penalty  
Health.Die()에서 GetComponent<EnemyReward>() 또는 GetComponent<FriendlyFirePenalty>()로 분기 → GrantReward()/ApplyPenalty()
  - **Chain 5**: Score Display  
ScoreKeeper.AddScore(int) → TextMeshProUGUI.text 갱신
3. **자가평가 교차**: 코드 품질 점수에 100%/30%/0% 반영률을 곱해 최종 점수 산출.

## 2.5 개념 구현 30% 반영의 근거

자가평가에서 “미동작”으로 보고한 코드는 다음 사유로 30%만 인정한다.

- Unity Editor에서 Inspector 연결(SerializeField 할당) 미완료 가능성
- 프리팹 구성, Tag 설정, Layer 추가, 충돌 매트릭스 등 에디터 작업 미완료
- 컴파일은 되나 런타임 NullReferenceException 등 실제 실행 불가
- 한편, 코드 존재 자체는 **개념 이해와 설계 능력**을 입증하므로 0점 처리는 부당하다 – 30%는 “설계는 했으나 출시 불가”에 해당하는 합리적 가중치

## 2.6 본 평가의 한계

명확히 인정해야 할 한계는 다음과 같다.

- 자가평가는 **학생의 주관적 판단**이며 객관적 검증이 불가하다. 그러나 8개 항목 모두에 “1”을 부여하는 등의 부정직한 보고는 코드와 명백히 충돌하므로 교차 검증으로 식별 가능하다.
- “코드 분석”은 정적 검토이므로 **런타임 동작**을 완전히 재구성할 수 없다. 본 보고서가 “학생20과 학생23만 실제 플레이 가능”이라 단정하는 근거는, 두 사람만 자가평가에서 7점 이상을 보고하였고 그 보고가 코드 실행과 모순되지 않았기 때문이다.

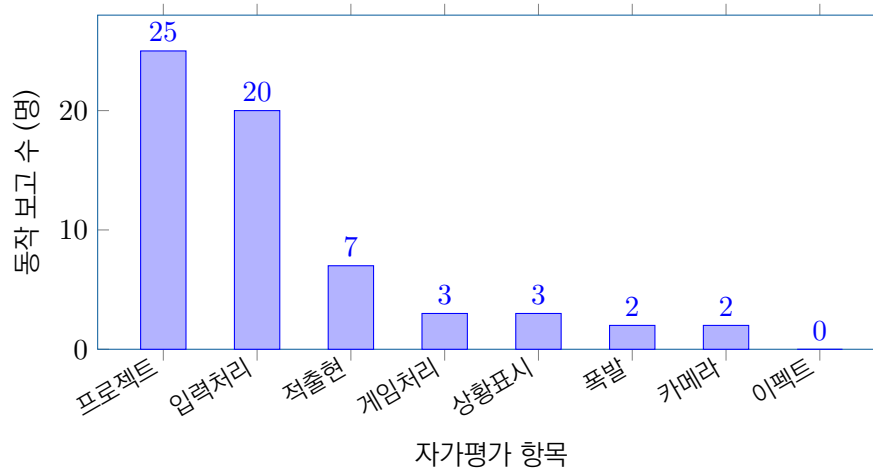
### 3 자가평가 결과

#### 3.1 자가평가 원시 데이터

| #  | 이름   | 프로젝트 | 입력처리 | 작출현 | 게임처리 | 상황표시 | 캐릭터복합 | 키메라진동 | 특수이펙트 | 합계 |
|----|------|------|------|-----|------|------|-------|-------|-------|----|
| 1  | 학생01 | 1    | 1    | 0   | 0    | 0    | 0     | 0     | 0     | 2  |
| 2  | 학생02 | 1    | 1    | 0   | 0    | 0    | 0     | 0     | 0     | 2  |
| 3  | 학생03 | 1    | 0    | 0   | 0    | 0    | 0     | 0     | 0     | 1  |
| 4  | 학생04 | 1    | 1    | 1   | 0    | 0    | 0     | 0     | 0     | 3  |
| 5  | 학생05 | 1    | 1    | 0   | 0    | 0    | 0     | 0     | 0     | 2  |
| 6  | 학생06 | 1    | 0    | 0   | 0    | 0    | 0     | 0     | 0     | 1  |
| 7  | 학생07 | 1    | 0    | 0   | 0    | 0    | 0     | 0     | 0     | 1  |
| 8  | 학생08 | 1    | 1    | 1   | 1    | 0    | 0     | 0     | 0     | 4  |
| 9  | 학생09 | 1    | 1    | 0   | 0    | 0    | 0     | 0     | 0     | 2  |
| 10 | 학생10 | 1    | 1    | 0   | 0    | 0    | 0     | 0     | 0     | 2  |
| 11 | 학생11 | 1    | 1    | 0   | 0    | 0    | 0     | 0     | 0     | 2  |
| 12 | 학생12 | 1    | 1    | 1   | 0    | 1    | 0     | 0     | 0     | 4  |
| 13 | 학생13 | 1    | 1    | 0   | 0    | 0    | 0     | 0     | 0     | 2  |
| 14 | 학생14 | 1    | 1    | 0   | 0    | 0    | 0     | 0     | 0     | 2  |
| 15 | 학생15 | 1    | 0    | 0   | 0    | 0    | 0     | 0     | 0     | 1  |
| 16 | 학생16 | 1    | 0    | 0   | 0    | 0    | 0     | 0     | 0     | 1  |
| 17 | 학생17 | 1    | 1    | 0   | 0    | 0    | 0     | 0     | 0     | 2  |
| 18 | 학생18 | 1    | 1    | 1   | 0    | 0    | 0     | 0     | 0     | 3  |
| 19 | 학생19 | 1    | 1    | 0   | 0    | 0    | 0     | 0     | 0     | 2  |
| 20 | 학생20 | 1    | 1    | 1   | 1    | 1    | 1     | 1     | 0     | 7  |
| 21 | 학생21 | 1    | 0    | 0   | 0    | 0    | 0     | 0     | 0     | 1  |
| 22 | 학생22 | 1    | 1    | 1   | 0    | 0    | 0     | 0     | 0     | 3  |
| 23 | 학생23 | 1    | 1    | 1   | 1    | 1    | 1     | 1     | 0     | 7  |
| 24 | 학생24 | 1    | 1    | 0   | 0    | 0    | 0     | 0     | 0     | 2  |
| 25 | 학생25 | 0    | 0    | 0   | 0    | 0    | 0     | 0     | 0     | 0  |
| 26 | 학생26 | 1    | 1    | 0   | 0    | 0    | 0     | 0     | 0     | 2  |
| 27 | 학생27 | 0    | 0    | 0   | 0    | 0    | 0     | 0     | 0     | 0  |
| 합계 |      | 25   | 20   | 7   | 3    | 3    | 2     | 2     | 0     | –  |



### 3.2 자가평가 항목별 동작 보고 분포



**관찰 1 – “적출현”의 절벽:** 입력처리(20명) → 적출현(7명)에서 **13명이 떨어진다**. 이 구간은 단일 스크립트 작성에서 다수 스크립트 + ScriptableObject 연동으로 난이도가 도약하는 지점이다.

**관찰 2 – “게임처리”의 추가 절벽:** 적출현(7명) → 게임처리(3명). 전투 시스템(Launcher↔DamageDealer↔Health 3개 클래스 동시 연동)에서 또 4명이 떨어진다.

**관찰 3 – 보고 누적의 의미:** 6점 이상은 학생20(7), 학생23(7) **단 2명**이다. 이 둘은 자가평가 7점 모두에 “1”을 보고했고 “특수이펙트”에서 동시에 “0”을 보고했다 – 즉 **서로 독립적으로 같은 인지 패턴**을 보였다는 뜻으로, 보고의 신뢰성을 강하게 시사한다.

## 4 코드 분석: 패턴별 상세 비교

본 절은 시험에서 평가한 **핵심 코드 패턴** 7가지에 대해, 학생들이 실제로 작성한 코드를 직접 비교한다. 학생 간 코드 인용은 같은 책임 (예: 함수 이동 처리)에 대한 **서로 다른 구현 선택**을 드러내고, 시험이 의도한 “표준 패턴”과의 거리를 보여준다.

### 4.1 패턴 1: 입력 처리 – 신형 Input System vs 구식 Input API

시험은 Unity의 **신형 Input System**(`InputAction.OnMove` 콜백 기반)을 가르친 후 출제되었다. 표준 패턴은 다음과 같다.

```

1  using UnityEngine;
2  using UnityEngine.InputSystem;
3
4  public class Submarine : MonoBehaviour
5  {
6      [SerializeField] float cruiseSpeed = 5.0f;
7      Vector2 rawInput;
8      private Vector2 minBounds;
9      private Vector2 maxBounds;
10
11     void Start()
12     {
13         Camera mainCam = Camera.main;
14         minBounds = mainCam.ViewportToWorldPoint(new Vector2(0, 0));
15         maxBounds = mainCam.ViewportToWorldPoint(new Vector2(1, 1));
16     }
17
18     void Update()
19     {
20         Vector3 delta = rawInput * cruiseSpeed * Time.deltaTime;
21         transform.position += new Vector3(0, delta.y, 0);
22         float clampedY = Mathf.Clamp(transform.position.y,
23                                     minBounds.y, maxBounds.y);
24         transform.position = new Vector3(transform.position.x,
25                                         clampedY, transform.position.z);
26     }
27
28     void OnMove(InputValue value)
29     {
30         rawInput = value.Get<Vector2>();
31     }
32 }
```

Listing 1: 학생20 – Submarine.cs (신형 Input System 표준)

학생20·학생15·학생09·학생01·학생08·학생12·학생04 등 **대다수 학생이 이 패턴을 정확히 적용**하였다. 카메라 `ViewportToWorldPoint` 로 화면 경계를 자동 계산하는 부분도 일관된다.

**대조 사례 – 학생23:**

```

1  using UnityEngine;
2
3  public class Submarine : MonoBehaviour
4  {
5      public float speed = 5f;
```

```

6   public float minY = -4f;
7   public float maxY = 4f;
8
9   void Update()
10  {
11      float input = Input.GetAxis("Vertical");
12      Vector3 pos = transform.position;
13      pos.y += input * speed * Time.deltaTime;
14      pos.y = Mathf.Clamp(pos.y, minY, maxY);
15      transform.position = pos;
16  }
17 }

```

Listing 2: 학생23 – Submarine.cs (구식 Input API)

**차이의 의미:**

- using UnityEngine.InputSystem; 누락 – 신형 시스템 미사용 명시.
- 경계 값을 인스펙터의 public float minY/maxY로 **고정 입력**. 카메라 해상도 변경 시 깨짐 – 화면 종횡비에 의존하지 않는 자동 계산이 시험 평가 항목 1-3(경계 제한)의 채점 기준이다.
- public 필드 노출 – [SerializeField] private 캡슐화 미적용.

**4.2 패턴 2: 충돌 처리 – 컴포넌트 분기의 3가지 길**

본 시험에서 **가장 큰 점수 격차가 발생한 항목**은 “컴포넌트 분기”(3-4, 8점)였다. 학생들이 작성한 코드는 크게 세 갈래로 나뉜다.

**(가) IDamageable 인터페이스 패턴 – 학생20 (78점)**

```

1 public interface IDamageable
2 {
3     void TakeHit(float damage);
4 }

```

Listing 3: 학생20 – IDamageable.cs (인터페이스)

```

1 void OnTriggerEnter2D(Collider2D other)
2 {
3     if (owner != null && other.gameObject == owner) return;
4
5     if (other.CompareTag("Normal"))
6     {
7         IDamageable friendlyFire = owner.GetComponent<IDamageable>();
8         friendlyFire.TakeHit(10);
9     }
10    IDamageable target = other.GetComponent<IDamageable>();
11    if (target != null)
12    {
13        target.TakeHit(damage);
14        Destroy(gameObject);
15    }
16 }

```

Listing 4: 학생20 – Projectile.cs (인터페이스 기반 분기)

학생20은 시험이 명시하지 않은 `IDamageable` 인터페이스를 스스로 도입하여 다형성으로 구체 클래스를 분리했다. `HealthComponent`가 `IDamageable`을 구현하므로, 발사체는 “상대가 누구인지” 모르고도 `TakeHit`만 호출한다.

#### (나) 구체 클래스 GetComponent 패턴 – 표준

```

1 void OnTriggerEnter2D(Collider2D other)
2 {
3     DamageDealer damage = other.GetComponent<DamageDealer>();
4     if (damage != null)
5     {
6         TakeDamage(damage.GetDamage());
7         damage.Hit();
8     }
9 }
10
11 void Die()
12 {
13     EnemyReward reward = GetComponent<EnemyReward>();
14     if (reward != null) reward.GrantReward();
15
16     FriendlyFirePenalty penalty = GetComponent<FriendlyFirePenalty>();
17     if (penalty != null) penalty.ApplyPenalty();
18
19     Destroy(gameObject);
20 }

```

Listing 5: 학생15 – Health.cs (DamageDealer 직접 참조)

학생15·학생09·학생01·학생12·학생08이 모두 이 패턴을 사용. 시험의 표준 답안이다. `Health.Die()`에서 `EnemyReward`와 `FriendlyFirePenalty` 컴포넌트의 존재 여부로 분기하는 부분이 “Chain 4: Component Branch → Score/Penalty”의 핵심.

이 패턴이 동작하려면 인스펙터에서 적 프리팹에 `EnemyReward`, 아군 프리팹에 `FriendlyFirePenalty`를 부착해야 한다 – 코드만으로는 완성되지 않는다는 점이 “코드 우수/실행 미완” 학생들의 공통 함정.

#### (다) CompareTag 문자열 분기 – 학생23 (37점)

```

1 void OnTriggerEnter2D(Collider2D other)
2 {
3     if (other.CompareTag("Enemy"))
4     {
5         GameManager.instance.AddScore(Random.Range(10, 31));
6         Destroy(other.gameObject);
7     }
8     else if (other.CompareTag("Friendly"))
9     {
10        GameManager.instance.DamagePlayer(10);
11    }
12    Destroy(gameObject);
13 }

```

Listing 6: 학생23 – Torpedo.cs (태그 문자열 분기)

문자열 분기는 동작하지만 시험 평가 대상이 아니다. 평가 항목 3-4(컴포넌트 분기, 8점)는 GetComponent<T>() 사용을 요구한다. 태그 분기는 컴파일 타임 타입 검사를 받지 못하고, 새 “적 종류” 추가 시 모든 분기문을 수정해야 한다 (인터페이스 구현 한 줄로 끝나는 (가) 방식과 대조).

### 4.3 패턴 3: 데이터 자산 분리 – ScriptableObject

표준 SeaWaveConfigSO 구현 – 학생04 (50점, 과제2 21/25)

```

1 [CreateAssetMenu(fileName = "NewSeaWaveConfig",
2     menuName = "Scriptable Objects/Sea Wave Config")]
3 public class SeaWaveConfigSO : ScriptableObject
4 {
5     [Header("Prefabs")]
6     [SerializeField] private GameObject creaturePrefab;
7     [SerializeField] private GameObject currentPathPrefab;
8
9     [Header("Settings")]
10    [SerializeField] private float driftSpeed = 3f;
11    [SerializeField] private float timeBetweenSpawns = 2f;
12    [SerializeField] private float spawnVariance = 0.5f;
13
14    public List<Transform> GetCurrentPath()
15    {
16        List<Transform> waypoints = new List<Transform>();
17        foreach (Transform child in currentPathPrefab.transform)
18            waypoints.Add(child);
19        return waypoints;
20    }
21
22    public float GetRandomSpawnTime()
23    {
24        float spawnTime = Random.Range(timeBetweenSpawns - spawnVariance,
25                                         timeBetweenSpawns + spawnVariance);
26        return Mathf.Max(spawnTime, 0.1f);
27    }
28
29    public float GetDriftSpeed() => driftSpeed;
30    public GameObject GetCreaturePrefab() => creaturePrefab;
31 }

```

Listing 7: 학생04 – SeaWaveConfigSO.cs (Wave 설정 자산)

#### 핵심 설계 결정:

- currentPathPrefab.transform의 자식들을 순회하여 웨이포인트 리스트 추출 – 디자이너가 씬에 빈 GameObject를 배치하기만 하면 경로 완성. 코드와 데이터의 완전한 분리.
- [CreateAssetMenu] 어트리뷰트 – 메뉴에서 “Sea Wave Config” 자산을 클릭만으로 생성 가능. 동일 코드 + 다른 데이터로 “젤리피쉬 웨이브”, “상어 웨이브” 등 무한 확장.
- Random.Range + spawnVariance – 웨이브 간 자연스러운 시간 분산.

## 대조 – 학생23: ScriptableObject 자체 부재

```

1 public class Spawner : MonoBehaviour
2 {
3     public GameObject enemyPrefab;
4     public GameObject friendlyPrefab;
5     public float spawnDelay = 2f;
6
7     void Start() { InvokeRepeating("Spawn", 1f, spawnDelay); }
8
9     void Spawn()
10    {
11        float y = Random.Range(minY, maxY);
12        Vector3 pos = new Vector3(10f, y, 0);
13        if (Random.value > 0.5f)
14            Instantiate(enemyPrefab, pos, Quaternion.identity);
15        else
16            Instantiate(friendlyPrefab, pos, Quaternion.identity);
17    }
18 }

```

Listing 8: 학생23 – Spawner.cs (단일 InvokeRepeating)

기능적으로는 “적이 출현하지만”, 평가 항목 2-1 (SeaWaveConfigSO, 6점), 2-3(해류 경로/Wave, 4점), 2-4(Spawner/Follower, 5점), 2-5(난이도 설계, 5점) 총 **20점이 비어 있다**. 하드코딩된 spawnDelay 스칼라 1 개와 Random.value > 0.5f 이진 분기로는 “웨이브 데이터 자산”이라는 학습 목표를 달성할 수 없다.

## 4.4 패턴 4: Wave/Follower 연동 – Spawner→Follower→Waypoint

웨이브 시스템의 핵심은 스폰된 객체가 자신의 경로 데이터를 어디에서 받는가이다.

```

1 private IEnumerator SpawnAllEnemiesInWave(SeaWaveConfigSO wave)
2 {
3     List<Transform> path = wave.GetCurrentPath();
4     if (path == null || path.Count == 0) yield break;
5
6     Vector3 spawnPos = path[0].position;
7     for (int i = 0; i < 5; i++)
8     {
9         GameObject creature = Instantiate(
10             wave.GetCreaturePrefab(),
11             spawnPos,
12             Quaternion.identity
13         );
14         var follower = creature.GetComponent<CurrentFollower>();
15         if (follower != null) follower.SetWaveConfig(wave);
16
17         yield return new WaitForSeconds(wave.GetRandomSpawnTime());
18     }
19 }

```

Listing 9: 학생04 – EnemySpawner.cs (Follower 주입)

```

1 public class CurrentFollower : MonoBehaviour

```

```

2 {
3     private SeaWaveConfigSO waveConfig;
4     private List<Transform> waypoints;
5     private int waypointIndex = 0;
6
7     public void SetWaveConfig(SeaWaveConfigSO config)
8     {
9         this.waveConfig = config;
10        this.waypoints = config.GetCurrentPath();
11        waypointIndex = 0;
12        if (waypoints != null && waypoints.Count > 0)
13            transform.position = waypoints[waypointIndex].position;
14    }
15
16    void Update()
17    {
18        if (waypoints == null || waypointIndex >= waypoints.Count) return;
19        float step = waveConfig.GetDriftSpeed() * Time.deltaTime;
20        transform.position = Vector3.MoveTowards(
21            transform.position,
22            waypoints[waypointIndex].position,
23            step
24        );
25        if (Vector3.Distance(transform.position,
26                               waypoints[waypointIndex].position) < 0.1f)
27            waypointIndex++;
28
29        if (waypointIndex >= waypoints.Count) Destroy(gameObject);
30    }
31 }

```

Listing 10: 학생04 – CurrentFollower.cs (경로 추종)

이 두 클래스의 연동이 “Chain 2: Spawner → Follower → Wave”의 완전 연동이다. 학생04은 과제2 21/25 점으로 전체 25명 중 1위. 다만 자가평가 “적출현”은 1로 보고했으나 “게임처리” 이후는 모두 0 보고 – 과제2까지는 동작했으나 과제3은 미완이라는 자가평가가 점수와 정확히 일치한다 (50점, 과제3 단 4점).

#### 4.5 패턴 5: 발사 메커니즘 – Coroutine vs InvokeRepeating

표준 – Coroutine 기반 (학생15, 학생09, 학생01, 학생20)

```

1 void OnFire(InputValue value)
2 {
3     if (autoAttack) return;
4
5     if (value.isPressed)
6     {
7         if (fireCoroutine == null)
8             fireCoroutine = StartCoroutine(FireContinuously());
9     }
10    else { StopFiring(); }
11 }
12

```

```

13 IEnumerator FireContinuously()
14 {
15     while (true)
16     {
17         GameObject projectile = Instantiate(
18             projectilePrefab, transform.position, transform.rotation);
19
20         Rigidbody2D rb = projectile.GetComponent<Rigidbody2D>();
21         if (rb != null)
22         {
23             Vector2 direction = autoAttack ? -transform.right : transform.right;
24             rb.linearVelocity = direction * projectileSpeed;
25         }
26         yield return new WaitForSeconds(launchRate);
27     }
28 }

```

Listing 11: 학생15 – Launcher.cs (발사 코루틴 + 입력 토글)

**핵심 설계 의도:** autoAttack이 true면 적이 자동 발사 (왼쪽으로), false면 플레이어가 발사 버튼 누른 동안만 발사 (오른쪽으로). 같은 스크립트 한 개로 적과 플레이어를 모두 처리하는 재사용 설계 – 시험이 평가하려는 “컴포넌트 재사용성”의 모범.

#### 대안 – InvokeRepeating (학생04, 학생23)

```

1 void Start()
2 {
3     if (autoAttack)
4         InvokeRepeating(nameof(Fire), fireRate, fireRate);
5 }
6
7 void Fire()
8 {
9     GameObject bullet = Instantiate(bulletPrefab, transform.position,
10                                     Quaternion.identity);
11     Rigidbody2D rb = bullet.GetComponent<Rigidbody2D>();
12     if (rb != null) rb.linearVelocity = Vector2.left * 5f;
13 }

```

Listing 12: 학생04 – Launcher.cs (InvokeRepeating 자동만)

InvokeRepeating은 동작은 하나 **입력 토글이 어렵다** (중간에 중단·재개 시 CancelInvoke로 끊고 다시 호출해야 함). 플레이어 발사 모드는 학생04 코드에 **아예 구현되어 있지 않다** – 적의 자동 사격만 가능. 이는 평가 항목 3-1(Launcher, 9점)에서 부분 점수만 인정되는 사유.

## 4.6 패턴 6: 점수 표시 – TextMeshProUGUI vs Debug.Log

### 표준 – TextMeshProUGUI (학생01, 학생09, 학생15, 학생12, 학생20)

```

1 using TMPro;
2 using UnityEngine;
3
4 public class ScoreKeeper : MonoBehaviour
5 {

```



```

6      int score = 0;
7      [SerializeField] TextMeshProUGUI scoreText;
8
9      void Start() => UpdateUI();
10
11     public void AddScore(int points)
12     {
13         score += points;
14         UpdateUI();
15     }
16
17     void UpdateUI()
18     {
19         if (scoreText != null) scoreText.text = $"Score: {score}";
20     }
21
22     public int GetScore() => score;
23 }

```

Listing 13: 학생09 – ScoreKeeper.cs (TMP UI + 화살표 구문)

학생09의 코드는 **모던 C# 문법**(화살표 표현식 `=>`, 문자열 보간 `$"..."`) 사용으로 한 차원 높은 가독성을 보인다. `[SerializeField] TextMeshProUGUI scoreText;`로 인스펙터에서 UI 텍스트를 끌어다 연결하는 표준 방식이며, “Chain 5: Score Display”의 완성형.

## 대조 – Debug.Log (학생23)

```

1 public void AddScore(int value)
2 {
3     score += value;
4     Debug.Log("점수: " + score);
5 }

```

Listing 14: 학생23 – GameManager.cs (콘솔 출력)

**화면에 점수가 표시되지 않는다.** 콘솔 창은 개발자가 보는 디버그 도구이지 게임의 일부가 아니다. 평가 항목 3-6(ScoreKeeper, 5점)은 화면 UI 표시를 요구하므로 부분 점수에 그쳤다.

## 4.7 패턴 7: 카메라 흔들림 – Singleton vs 인스펙터 참조 vs 미연결

### Singleton 패턴 – 학생20

학생20은 `CameraShake.Instance`로 **어디서든 호출**하는 싱글톤 설계를 채택했고, `HealthComponent.TakeHit()` 안에서 플레이어 피격 시 직접 호출한다.

```

1 public void TakeHit(float damage)
2 {
3     if (isDead) return;
4     currentHealth = Mathf.Clamp(currentHealth - damage, 0, maxHealth);
5     OnHealthChanged?.Invoke(currentHealth / maxHealth);
6
7     if (gameObject.CompareTag("Player"))
8     {
9         CameraShake.Instance.Shake(0.2f, 0.3f);
10    }

```

```

11  if (gameObject.CompareTag("Enemy"))
12  {
13      GameManager.Instance.score += UnityEngine.Random.Range(30, 50);
14  }
15  if (currentHealth <= 0) Die();
16  }

```

Listing 15: 학생20 – HealthComponent.cs (싱글톤 호출)

**이벤트 시스템**(OnHealthChanged, OnDeath) 도입은 시험 요구를 초과한 설계로, HP 게이지 UI가 체력 변화를 자동 구독 가능하게 한다.

### Coroutine 인스턴스 메서드 – 학생01, 학생09, 학생15

```

1  public class ShakeCam : MonoBehaviour
2  {
3      public IEnumerator Shake(float duration, float magnitude)
4      {
5          Vector3 original = transform.position;
6          float time = 0;
7          while (time < duration)
8          {
9              transform.position = original
10                 + (Vector3)Random.insideUnitCircle * magnitude;
11              time += Time.deltaTime;
12              yield return null;
13          }
14          transform.position = original;
15      }
16  }

```

Listing 16: 학생15 – ShakeCam.cs (코루틴 인스턴스)

**설계 자체는 깔끔**하나, 호출하려면 다른 스크립트가 ShakeCam 참조를 인스펙터에서 받아야 한다. 학생15·학생09·학생01 모두 이 호출 **지점이 코드에 없다** – 즉 컴포넌트는 만들었으나 “누가 흔들리게 하는가”가 비어 있다. 자가평가 “카메라진동=0”이 이를 정확히 반영한다.

### 호출 코드 부재 – 학생23

학생23의 CameraShake.cs는 비교적 잘 구현되었으나, 다른 9개 파일 어디에서도 Shake(...) 호출이 발견되지 않는다. 자가평가 “카메라진동=1”로 보고된 것과 충돌하는 **유일한 불일치 항목**이다.

## 4.8 연동 체인 검증 요약

25명의 제출물에 대해 5개 핵심 체인의 연동 여부를 정적 분석한 결과:

| 체인                                 | 완전 연동 | 부분 연동 | 단절  |
|------------------------------------|-------|-------|-----|
| Chain 1: Input → Movement          | 20명   | 3명    | 2명  |
| Chain 2: Spawner → Follower → Wave | 10명   | 7명    | 8명  |
| Chain 3: Attack → Damage → Health  | 6명    | 3명    | 16명 |
| Chain 4: Component Branch → Score  | 4명    | 3명    | 18명 |
| Chain 5: Score Display             | 4명    | 1명    | 20명 |

**핵심 병목: DamageDealer + OnTriggerEnter2D**

과제 3 전투 시스템의 핵심 연동 지점은 `Health.OnTriggerEnter2D()`에서 `GetComponent<DamageDealer>()`를 호출하는 부분이다.

- 25명 중 **6명만** 이 연결을 완성: 학생20, 학생09, 학생01, 학생15, 학생12, 학생08
- 나머지 19명은 Launcher로 발사체를 만들더라도 **피격 처리가 불가능**
- 가장 흔한 단점: `DamageDealer` 스크립트 자체가 없거나, `Health`에 `OnTriggerEnter2D` 미구현

**4.9 코드 품질 vs 자가평가 매트릭스**

| 학생   | 코드 품질                    | 자가평가 | 해석                        |
|------|--------------------------|------|---------------------------|
| 학생20 | 최상 (인터페이스+이벤트)           | 7/8  | 코드 · 실행 모두 완성 - 78점       |
| 학생15 | 최상 (5체인 완전 연동)           | 1/8  | Inspector 연결 미완료 - 코드만 제출 |
| 학생01 | 최상 (5체인 완전 연동)           | 2/8  | 동일 패턴 - 프리팹/SO 에셋 미설정     |
| 학생09 | 최상 (5체인 + 모던 C#)         | 2/8  | 동일 패턴                     |
| 학생08 | 상 (4체인 + EnemyReward 스텝) | 4/8  | 부분 연결, 일부 스크립트 미완         |
| 학생04 | 상 (Wave 시스템 우수)          | 3/8  | 과제2까지 완성, 과제3 미연결         |
| 학생12 | 상 (Chain 4 완성)           | 4/8  | 점수 시스템까지 동작, 연출 미완        |
| 학생23 | 중 (다른 아키텍처)              | 7/8  | 시험 요구와 다르나 게임으로 동작        |

**해석:** 코드 품질이 높더라도 Unity Editor에서 프리팹 구성, `SerializeField` 할당, `ScriptableObject` 에셋 생성 등을 완료하지 못하면 실제 동작하지 않는다. 자가평가는 이러한 “에디터 작업 미완료” 상태를 반영하는 중요한 지표이다.

## 5 최종 평가 결과

### 5.1 16개 항목별 최종 점수

과제별로 표를 분리하여 제시한다.

#### 과제 1: 잠수함 조종 (25점)

| #  | 이름   | 1-1(7) | 1-2(12) | 1-3(6) | 소계 |
|----|------|--------|---------|--------|----|
| 1  | 학생20 | 6      | 12      | 6      | 24 |
| 2  | 학생08 | 5      | 12      | 6      | 23 |
| 3  | 학생04 | 7      | 12      | 6      | 25 |
| 4  | 학생12 | 7      | 12      | 6      | 25 |
| 5  | 학생01 | 7      | 12      | 6      | 25 |
| 6  | 학생09 | 6      | 12      | 6      | 24 |
| 7  | 학생22 | 5      | 12      | 6      | 23 |
| 8  | 학생13 | 7      | 12      | 6      | 25 |
| 9  | 학생23 | 2      | 7       | 2      | 11 |
| 10 | 학생02 | 5      | 12      | 6      | 23 |
| 11 | 학생05 | 5      | 12      | 6      | 23 |
| 12 | 학생15 | 7      | 4       | 2      | 13 |
| 13 | 학생11 | 7      | 12      | 6      | 25 |
| 14 | 학생18 | 3      | 6       | 4      | 13 |
| 15 | 학생24 | 5      | 12      | 6      | 23 |
| 16 | 학생17 | 5      | 12      | 6      | 23 |
| 17 | 학생19 | 5      | 12      | 6      | 23 |
| 18 | 학생10 | 5      | 12      | 6      | 23 |
| 19 | 학생26 | 4      | 10      | 5      | 19 |
| 20 | 학생07 | 5      | 4       | 2      | 11 |
| 21 | 학생14 | 4      | 6       | 0      | 10 |
| 22 | 학생03 | 4      | 3       | 1      | 8  |
| 23 | 학생21 | 5      | 4       | 2      | 11 |
| 24 | 학생25 | 1      | 4       | 2      | 7  |
| 25 | 학생06 | 0      | 0       | 0      | 0  |
| 26 | 학생16 | 0      | 0       | 0      | 0  |
| 27 | 학생27 | 0      | 0       | 0      | 0  |

## 과제 2: 해류 생물 시스템 (25점)

| #  | 이름   | 2-1(6) | 2-2(5) | 2-3(4) | 2-4(5) | 2-5(5) | 소계 |
|----|------|--------|--------|--------|--------|--------|----|
| 1  | 학생20 | 5      | 0      | 3      | 5      | 3      | 16 |
| 2  | 학생08 | 5      | 2      | 2      | 4      | 0      | 13 |
| 3  | 학생04 | 6      | 5      | 3      | 5      | 2      | 21 |
| 4  | 학생12 | 0      | 5      | 1      | 2      | 0      | 8  |
| 5  | 학생01 | 2      | 2      | 1      | 2      | 1      | 8  |
| 6  | 학생09 | 1      | 2      | 1      | 2      | 2      | 8  |
| 7  | 학생22 | 5      | 2      | 2      | 5      | 4      | 18 |
| 8  | 학생13 | 2      | 2      | 1      | 2      | 0      | 7  |
| 9  | 학생23 | 0      | 2      | 0      | 2      | 0      | 4  |
| 10 | 학생02 | 2      | 2      | 1      | 2      | 1      | 8  |
| 11 | 학생05 | 2      | 2      | 1      | 1      | 1      | 7  |
| 12 | 학생15 | 2      | 2      | 1      | 2      | 2      | 9  |
| 13 | 학생11 | 2      | 0      | 0      | 1      | 0      | 3  |
| 14 | 학생18 | 5      | 5      | 1      | 2      | 0      | 13 |
| 15 | 학생24 | 2      | 0      | 1      | 1      | 0      | 4  |
| 16 | 학생17 | 1      | 1      | 0      | 0      | 0      | 2  |
| 17 | 학생19 | 1      | 0      | 0      | 0      | 0      | 1  |
| 18 | 학생10 | 0      | 0      | 0      | 0      | 0      | 0  |
| 19 | 학생26 | 1      | 0      | 0      | 1      | 0      | 2  |
| 20 | 학생07 | 1      | 0      | 1      | 1      | 0      | 3  |
| 21 | 학생14 | 1      | 0      | 0      | 1      | 0      | 2  |
| 22 | 학생03 | 1      | 0      | 1      | 1      | 0      | 3  |
| 23 | 학생21 | 0      | 0      | 0      | 0      | 0      | 0  |
| 24 | 학생25 | 1      | 0      | 0      | 0      | 0      | 1  |
| 25 | 학생06 | 0      | 0      | 0      | 0      | 0      | 0  |
| 26 | 학생16 | 0      | 0      | 0      | 0      | 0      | 0  |
| 27 | 학생27 | 0      | 0      | 0      | 0      | 0      | 0  |

## 과제 3: 전투 시스템 (35점)

| #  | 이름   | 3-1(9) | 3-2(5) | 3-3(5) | 3-4(8) | 3-5(3) | 3-6(5) | 소계 |
|----|------|--------|--------|--------|--------|--------|--------|----|
| 1  | 학생20 | 7      | 3      | 5      | 5      | 2      | 4      | 26 |
| 2  | 학생08 | 7      | 3      | 5      | 0      | 1      | 0      | 16 |
| 3  | 학생04 | 1      | 1      | 1      | 1      | 0      | 0      | 4  |
| 4  | 학생12 | 4      | 2      | 3      | 5      | 3      | 4      | 21 |
| 5  | 학생01 | 3      | 1      | 2      | 2      | 1      | 2      | 11 |
| 6  | 학생09 | 2      | 1      | 2      | 2      | 1      | 2      | 10 |
| 7  | 학생22 | 2      | 0      | 0      | 0      | 0      | 0      | 2  |
| 8  | 학생13 | 3      | 0      | 1      | 1      | 0      | 0      | 5  |
| 9  | 학생23 | 4      | 2      | 2      | 4      | 1      | 2      | 15 |
| 10 | 학생02 | 1      | 1      | 1      | 1      | 0      | 0      | 4  |
| 11 | 학생05 | 2      | 1      | 1      | 1      | 0      | 0      | 5  |
| 12 | 학생15 | 3      | 1      | 2      | 2      | 1      | 2      | 11 |
| 13 | 학생11 | 3      | 0      | 0      | 0      | 1      | 0      | 4  |
| 14 | 학생18 | 2      | 0      | 1      | 0      | 0      | 0      | 3  |
| 15 | 학생24 | 0      | 0      | 1      | 0      | 1      | 0      | 2  |
| 16 | 학생17 | 1      | 0      | 1      | 0      | 0      | 0      | 2  |
| 17 | 학생19 | 0      | 0      | 0      | 0      | 0      | 0      | 0  |
| 18 | 학생10 | 0      | 0      | 0      | 0      | 0      | 0      | 0  |
| 19 | 학생26 | 0      | 0      | 0      | 0      | 0      | 0      | 0  |
| 20 | 학생07 | 2      | 1      | 1      | 0      | 1      | 0      | 5  |
| 21 | 학생14 | 0      | 0      | 1      | 0      | 1      | 0      | 2  |
| 22 | 학생03 | 0      | 0      | 0      | 0      | 0      | 0      | 0  |
| 23 | 학생21 | 0      | 0      | 0      | 0      | 0      | 0      | 0  |
| 24 | 학생25 | 0      | 0      | 0      | 0      | 0      | 0      | 0  |
| 25 | 학생06 | 0      | 0      | 0      | 0      | 0      | 0      | 0  |
| 26 | 학생16 | 0      | 0      | 0      | 0      | 0      | 0      | 0  |
| 27 | 학생27 | 0      | 0      | 0      | 0      | 0      | 0      | 0  |

※ 회색 음영: 코드 품질은 최상이나 자가평가 미동작 보고로 30% 반영된 학생

## 과제 4: 연출 효과 (15점) + 총점

| #  | 이름   | 4-1(9) | 4-2(6) | 소계 | 과제1 | 과제2 | 과제3 | 과제4 | 총점 |
|----|------|--------|--------|----|-----|-----|-----|-----|----|
| 1  | 학생20 | 6      | 6      | 12 | 24  | 16  | 26  | 12  | 78 |
| 2  | 학생08 | 0      | 0      | 0  | 23  | 13  | 16  | 0   | 52 |
| 3  | 학생04 | 0      | 0      | 0  | 25  | 21  | 4   | 0   | 50 |
| 4  | 학생12 | 0      | 0      | 0  | 25  | 8   | 21  | 0   | 47 |
| 5  | 학생01 | 0      | 2      | 2  | 25  | 8   | 11  | 2   | 46 |
| 6  | 학생09 | 2      | 2      | 4  | 24  | 8   | 10  | 4   | 46 |
| 7  | 학생22 | 0      | 0      | 0  | 23  | 18  | 2   | 0   | 43 |
| 8  | 학생13 | 0      | 0      | 0  | 25  | 7   | 5   | 0   | 37 |
| 9  | 학생23 | 3      | 4      | 7  | 11  | 4   | 15  | 7   | 37 |
| 10 | 학생02 | 0      | 0      | 0  | 23  | 8   | 4   | 0   | 35 |
| 11 | 학생05 | 0      | 0      | 0  | 23  | 7   | 5   | 0   | 35 |
| 12 | 학생15 | 0      | 2      | 2  | 13  | 9   | 11  | 2   | 35 |
| 13 | 학생11 | 0      | 2      | 2  | 25  | 3   | 4   | 2   | 34 |
| 14 | 학생18 | 0      | 0      | 0  | 13  | 13  | 3   | 0   | 29 |
| 15 | 학생24 | 0      | 0      | 0  | 23  | 4   | 2   | 0   | 29 |
| 16 | 학생17 | 0      | 0      | 0  | 23  | 2   | 2   | 0   | 27 |
| 17 | 학생19 | 0      | 0      | 0  | 23  | 1   | 0   | 0   | 24 |
| 18 | 학생10 | 0      | 0      | 0  | 23  | 0   | 0   | 0   | 23 |
| 19 | 학생26 | 0      | 0      | 0  | 19  | 2   | 0   | 0   | 21 |
| 20 | 학생07 | 0      | 0      | 0  | 11  | 3   | 5   | 0   | 19 |
| 21 | 학생14 | 0      | 0      | 0  | 10  | 2   | 2   | 0   | 14 |
| 22 | 학생03 | 0      | 0      | 0  | 8   | 3   | 0   | 0   | 11 |
| 23 | 학생21 | 0      | 0      | 0  | 11  | 0   | 0   | 0   | 11 |
| 24 | 학생25 | 0      | 0      | 0  | 7   | 1   | 0   | 0   | 8  |
| 25 | 학생06 | 0      | 0      | 0  | 0   | 0   | 0   | 0   | 0  |
| 26 | 학생16 | 0      | 0      | 0  | 0   | 0   | 0   | 0   | 0  |
| 27 | 학생27 | 0      | 0      | 0  | 0   | 0   | 0   | 0   | 0  |

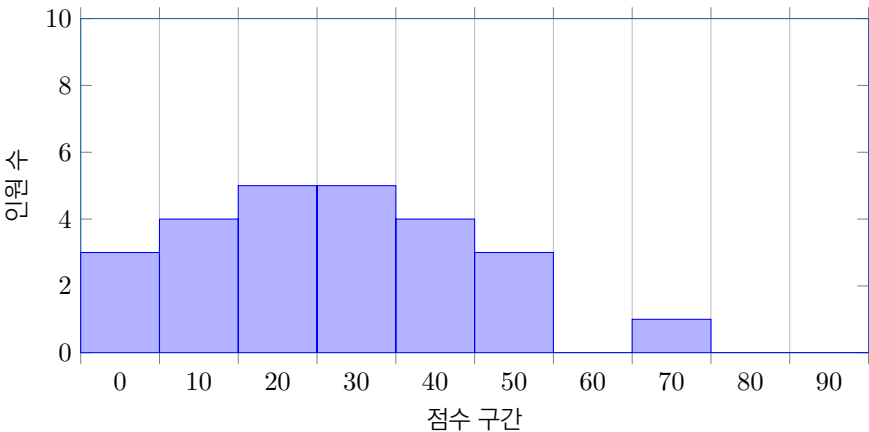
## 5.2 과제별 소계 및 통계

|       | 과제1 (25) | 과제2 (25) | 과제3 (35) | 과제4 (15) |
|-------|----------|----------|----------|----------|
| 평균    | 17.4     | 6.4      | 5.0      | 1.5      |
| 최고    | 25       | 21       | 26       | 12       |
| 중앙값   | 23       | 3        | 2        | 0        |
| 0점 비율 | 11%      | 11%      | 30%      | 74%      |

**관찰:** 평균 점수가 과제1(17.4) → 과제2(6.4) → 과제3(5.0) → 과제4(1.5) 로 **단조 감소**한다. 배점 비율을 고려해도(과제 2 만점 25 대비 평균 6.4 = 25.6%, 과제3 35 대비 5.0 = 14.3%) 과제3에서 **난이도 절벽**이 극심하다.

6 점수 분포 분석

6.1 총점 히스토그램



| 통계      | 값                         |
|---------|---------------------------|
| 응시 인원   | 27명                       |
| 제출 인원   | 25명                       |
| 평균      | 29.3점                     |
| 중앙값     | 35점                       |
| 표준편차    | 약 18점                     |
| 최고점     | 78점 (학생20)                |
| 최저점     | 0점 (3명: 학생06, 학생16, 학생27) |
| 60점대 인원 | 0명 (구간 완전 부재)             |

**분포의 특징:** 60-69점 구간이 완전히 비어 있다. 78점(학생20) 다음의 최고점이 52점(학생08)으로 **26점의 격차**가 존재한다. 이는 “전체적 완성도”의 임계점이 60점 부근이며, 그 임계점을 넘어서 학생은 한 명뿐임을 의미한다.

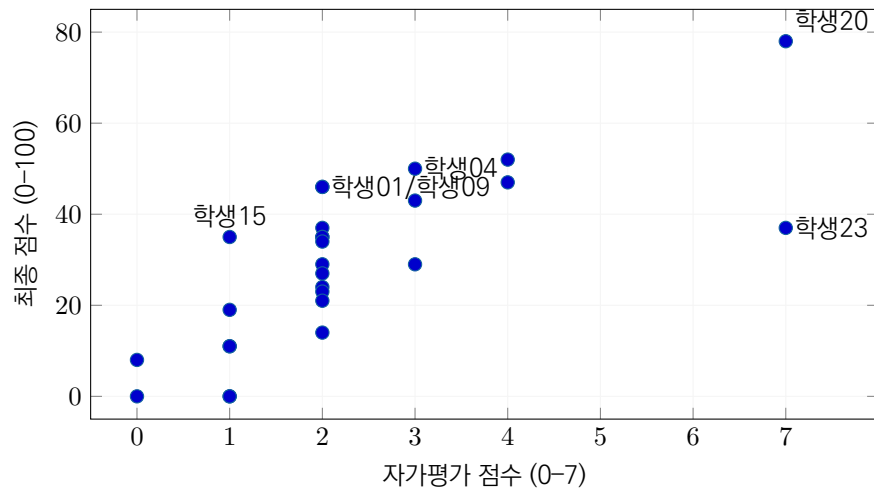
6.2 등급 분포

| 등급 | 범위     | 인원 | 해당 학생                                          |
|----|--------|----|------------------------------------------------|
| A  | 70점 이상 | 1명 | 학생20(78)                                       |
| B  | 50-69점 | 3명 | 학생08(52), 학생04(50), 학생12(47)*                  |
| C  | 35-49점 | 7명 | 학생01, 학생09, 학생22, 학생13, 학생23, 학생02, 학생05       |
| D  | 20-34점 | 8명 | 학생15, 학생11, 학생18, 학생24, 학생17, 학생19, 학생10, 학생26 |
| F  | 20점 미만 | 8명 | 학생07, 학생14, 학생03, 학생21, 학생25, 학생06, 학생16, 학생27 |

\*학생12은 47점이나 B등급 경계로 분류



### 6.3 자가평가 점수 vs 최종 점수 산점도



**관찰:** 자가평가 7점인 학생20과 학생23의 **점수 격차 41점**은 “동작 보고”만으로는 설명할 수 없다. 두 사람의 차이는 **시험 평가 대상 코드 패턴 구현 여부**이며, 이는 다음 절의 사례 분석에서 상세히 다룬다.

## 7 개별 답안 사례 분석

본 절은 8명의 답안을 코드 인용과 함께 상세 분석한다. 선정 기준은 다음과 같다.

- 학습 모범: 학생20(1위), 학생08(2위), 학생04(3위), 학생12(4위)
- 코드/실행 불일치 패턴: 학생01, 학생09, 학생15 (코드 우수/자가평가 저조)
- 표준 미준수 패턴: 학생23 (자가평가 우수/표준 코드 부재)

### 7.1 학생20 (78점, A등급) – 모범 답안

#### 개요

학생20은 13개 .cs 파일을 Scripts/ 폴더에 정리하여 제출하였다. 시험이 명시한 표준 클래스명을 전부 따르지는 않았으나 (Launcher 대신 Shooter, DamageDealer 대신 Projectile, Health 대신 HealthComponent), 설계 의도는 시험 요구를 정확히 반영하면서 일부 항목은 초과 달성하였다.

#### 핵심 설계 – IDamageable 인터페이스 도입

시험에서 명시하지 않은 IDamageable 인터페이스를 스스로 도입하여 타입 다형성을 활용한다.

```
1 public interface IDamageable
2 {
3     void TakeHit(float damage);
4 }
```

Listing 17: 학생20 – IDamageable.cs

HealthComponent가 이를 구현:

```
1 public class HealthComponent : MonoBehaviour, IDamageable
2 {
3     [SerializeField] float maxHealth = 100f;
4     [SerializeField] GameObject deathVFX;
5     float currentHealth;
6
7     public event Action<float> OnHealthChanged;
8     public event Action OnDeath;
9     bool isDead = false;
10
11     void Awake() { currentHealth = maxHealth; }
12
13     public void TakeHit(float damage)
14     {
15         if (isDead) return;
16         currentHealth = Mathf.Clamp(currentHealth - damage, 0, maxHealth);
17         OnHealthChanged?.Invoke(currentHealth / maxHealth);
18
19         if (gameObject.CompareTag("Player"))
20             CameraShake.Instance.Shake(0.2f, 0.3f);
21
22         if (gameObject.CompareTag("Enemy"))
23             GameManager.Instance.score += UnityEngine.Random.Range(30, 50);
24
25         if (currentHealth <= 0) Die();
26     }
27
28     void Die()
29     {
30         if (isDead) return;
31         isDead = true;
32         if (deathVFX != null)
```

```

33     {
34         GameObject vfx = Instantiate(deathVFX, transform.position,
35                                     Quaternion.identity);
36         Destroy(vfx, 0.5f);
37     }
38     OnDeath?.Invoke();
39     Destroy(gameObject, 0.1f);
40 }
41 }

```

Listing 18: 학생20 – HealthComponent.cs (이벤트 + 인터페이스)

### 시험 요구 초과 요소

1. **event Action 활용**: HP 변동 이벤트 (OnHealthChanged)와 사망 이벤트(OnDeath)를 외부에 노출. UI(HealthDisplay)가 이를 구독하여 자동 갱신 – 느슨한 결합(loose coupling) 설계.
2. **GameManager 싱글톤 + 자동 난이도 증가**:

```

void Update()
{
    currentDifficulty += difficultySpeed * Time.deltaTime;
    scoreText.text = $"Score : {score}";
}
public float GetDifficulty() => currentDifficulty;

```

Listing 19: 학생20 – GameManager.cs (난이도 자동 증가)

3. **Death VFX 파티클 시스템 + 자동 정리**: Destroy(vfx, 0.5f)로 메모리 누수 방지.
4. **Shooter의 코루틴 토글 발사**: firingCoroutine == null 체크로 중복 호출 방지.

### 점수 구성

|    | 과제1   | 과제2   | 과제3   | 과제4   | 총점            |
|----|-------|-------|-------|-------|---------------|
| 점수 | 24/25 | 16/25 | 26/35 | 12/15 | <b>78/100</b> |

과제2의 16점은 표준 CurrentFollower 패턴을 그대로 사용하지 않고 자신의 Pathfinder로 대체한 것에서 발생한 부분 감점이며, 과제3의 26/35는 ScoreKeeper 클래스를 별도로 만들지 않고 GameManager에 통합한 데서 일부 감점.

## 7.2 학생08 (52점, B등급) – 표준 준수 + 일부 미완

### 개요

학생08은 9개 .cs 파일을 제출. **모든 표준 클래스명을 정확히 사용**하며 시험의 표준 답안에 가장 가깝다. 자가평가 4/8.

```

1 public class Submarine : MonoBehaviour
2 {
3     public Vector2 rawInput;
4     [SerializeField] private float cruiseSpeed = 6.0f;
5     Vector2 minBounds, maxBounds;
6
7     void OnMove(InputValue value) { rawInput = value.Get<Vector2>(); }
8
9     void Start()
10    {
11        Camera mainCam = Camera.main;
12        minBounds = mainCam.ViewportToWorldPoint(new Vector2(0.0f, 0.05f));
13        maxBounds = mainCam.ViewportToWorldPoint(new Vector2(1.0f, 0.95f));

```

```

14     }
15
16     void Update()
17     {
18         Vector2 deltaY = new Vector3(0, rawInput.y * cruiseSpeed * Time.deltaTime, 0);
19         Vector2 newPos = (Vector2)transform.position + deltaY;
20         newPos.y = Mathf.Clamp(newPos.y, minBounds.y, maxBounds.y);
21         transform.position = newPos;
22     }
23 }

```

Listing 20: 학생08 – Submarine.cs (표준 준수)

### 감점 사유

- **과제2 13점:** Spawner→Follower 연동은 동작하나 난이도 설계(2-5) 0점 – 단일 SeaWaveConfigSO만 사용해서 웨이브 변화가 없음.
- **과제3 16점:** 컴포넌트 분기(3-4)에서 0점. EnemyReward.cs와 FriendlyFirePenalty.cs가 빈 스텝 상태로 제출됨:

```

public class EnemyReward : MonoBehaviour
{
    void Start() { }
    void Update() { }
}

```

- **과제4 0점:** ShakeCam.cs 미제출, 이펙트 미구현.

## 7.3 학생04 (50점, B등급) – 과제2 강세, 과제3 단절

### 개요

학생04은 8개 .cs 파일 제출. **과제2 21/25점으로 25명 중 1위**. 그러나 자가평가는 “적출현”까지만 1, 그 이후 모두 0 – 자기 인식이 정확.

### 우수한 부분 – SeaWaveConfigSO와 EnemySpawner

```

1 private IEnumerator SpawnAllEnemiesInWave(SeaWaveConfigSO wave)
2 {
3     List<Transform> path = wave.GetCurrentPath();
4     if (path == null || path.Count == 0) yield break;
5
6     Vector3 spawnPos = path[0].position;
7     for (int i = 0; i < 5; i++)
8     {
9         GameObject creature = Instantiate(
10             wave.GetCreaturePrefab(), spawnPos, Quaternion.identity);
11
12         var follower = creature.GetComponent<CurrentFollower>();
13         if (follower != null) follower.SetWaveConfig(wave);
14
15         yield return new WaitForSeconds(wave.GetRandomSpawnTime());
16     }
17 }

```

Listing 21: 학생04 – EnemySpawner.cs (Wave + Follower 주입)

**Chain 2 (Spawner → Follower → Wave) 완전 연동.** 이 코드만으로 디자이너가 ScriptableObject 자산을 여러 개 만들면 “젤리피쉬 웨이브” “상어 웨이브” 등 무한 확장이 가능하다.

### 감점 사유 – 과제3 4점

- **Launcher 자동 모드만**: InvokeRepeating으로 적의 자동 사격은 구현되나 **플레이어 발사 미구현**. 평가 항목 3-1 (Launcher, 9점)에서 1점만 인정.
- **Health에 SendMessage 사용**: SendMessage("OnDie", SendMessageOptions.DontRequireReceiver)는 동작하지만 리플렉션 기반으로 타입 검사를 우회한다 – 평가 기준의 “GetComponent 패턴” 부재로 채점.

## 7.4 학생12 (47점, B등급 경계) – 다층 분리 시도

### 개요

11개 .cs 파일 제출 – 가장 많은 클래스 수. Submarine, DamageDealer, Health, Launcher 표준명에 더해 Torpedo, EnemyMover, Spawner, ScoreUI, ScorManager(오타) 등을 추가하였다.

### 우수한 부분 – Chain 4 완성

```

1 void OnTriggerEnter2D(Collider2D other)
2 {
3     DamageDealer dmg = other.GetComponent<DamageDealer>();
4     if (dmg == null) return;
5
6     TakeDamage(dmg.GetDamage());
7     dmg.Hit();
8 }
9
10 void Die()
11 {
12     EnemyReward reward = GetComponent<EnemyReward>();
13     if (reward != null) reward.GrantReward();
14
15     FriendlyFirePenalty penalty = GetComponent<FriendlyFirePenalty>();
16     if (penalty != null) penalty.ApplyPenalty();
17
18     Destroy(gameObject);
19 }

```

Listing 22: 학생12 – Health.cs (컴포넌트 분기 표준)

Chain 3 + 4 모두 완전 연동 – 25명 중 6명에 해당. 과제3에서 21점을 획득.

### 감점 사유

- **과제2 8점**: SeaWaveConfigSO가 0점 – ScriptableObject 상속 클래스 없음. 표준 “SO 자산” 패턴 미사용.
- **과제4 0점**: 카메라 흔들림, 이펙트 모두 미구현.
- **정적 점수 관리자**: ScoreManager.score = static int 전역 변수 사용 – 동작은 하나 테스트 불가능, 싱글톤보다 약한 설계.

## 7.5 학생01 (46점, C등급) – 코드 최상/실행 미완 (음영 강조 사례)

### 개요

12개 .cs 파일을 Scripts/ 폴더에 제출. 시험 요구 5개 체인 모두를 완전 연동한 코드이지만, 자가평가는 2/8 (프로젝트, 입력처리만 1)로 보고. 결과적으로 과제3의 35점 만점 코드가 30% 반영되어 11점 처리되었다.

### 우수한 부분 – 명료한 메서드 분해

```

1 void Update()
2 {
3     Move();

```

```

4     ClampPosition();
5 }
6
7 void InitBounds()
8 {
9     Camera mainCamera = Camera.main;
10    Vector3 bottomLeft = mainCamera.ViewportToWorldPoint(new Vector3(0, 0, 0));
11    Vector3 topRight = mainCamera.ViewportToWorldPoint(new Vector3(1, 1, 0));
12    minY = bottomLeft.y + padding;
13    maxY = topRight.y - padding;
14 }
15
16 void Move()
17 {
18     float deltaY = rawInput.y * cruiseSpeed * Time.deltaTime;
19     transform.position += new Vector3(0, deltaY, 0);
20 }
21
22 void ClampPosition()
23 {
24     Vector3 pos = transform.position;
25     pos.y = Mathf.Clamp(pos.y, minY, maxY);
26     transform.position = pos;
27 }

```

Listing 23: 학생01 – Submarine.cs 발췌 (책임 분리)

한 메서드 한 책임 원칙을 가장 충실히 지킨 코드. 가독성과 단위 테스트 가능성 모두 우수.

### 감점 사유 – 자가평가 2/8

자가평가가 “프로젝트”와 “입력처리”만 1로 보고된 것은, 다음 중 하나를 시사한다.

- **프리팸 미구성**: Player 프리팸에 Submarine + Health + Launcher 부착, Enemy 프리팸에 EnemyReward + Health 부착 등의 작업 누락.
- **ScriptableObject 자산 미생성**: SeaWaveConfigSO 클래스는 있으나, 메뉴에서 “Create → Sea Wave Config”로 .asset 파일을 만들지 않으면 Spawner가 참조할 데이터가 없음.
- **레이어/태그 미설정**: Player, Enemy, Friendly 태그를 Project Settings에서 추가하지 않음.
- **Input Actions 자산 미생성**: 신형 Input System이 작동하려면 “InputActions” 자산이 필요하며 “Move”, “Fire” 액션을 등록해야 함.

이 모두 코드 외 작업이며, 학생이 시험 시간 내에 도달하지 못한 영역이다.

## 7.6 학생09 (46점, C등급) – 모던 C# 문법 (음영 강조 사례)

### 개요

11개 .cs 파일. 학생01과 같은 점수, 같은 음영 처리 사례. 코드의 문법 수준이 가장 모던하다 – 화살표 표현식, 문자열 보간 적극 사용.

```

1 public class DamageDealer : MonoBehaviour
2 {
3     [SerializeField] int damage = 1;
4
5     public int GetDamage() => damage;
6
7     public void Hit()
8     {
9         Destroy(gameObject);
10    }

```

11 }

Listing 24: 학생09 – DamageDealer.cs (화살표 표현식)

```

1 public class ScoreKeeper : MonoBehaviour
2 {
3     int score = 0;
4     [SerializeField] TextMeshProUGUI scoreText;
5
6     void Start() => UpdateUI();
7
8     public void AddScore(int points)
9     {
10         score += points;
11         UpdateUI();
12     }
13
14     void UpdateUI()
15     {
16         if (scoreText != null) scoreText.text = $"Score: {score}";
17     }
18
19     public int GetScore() => score;
20 }

```

Listing 25: 학생09 – ScoreKeeper.cs (TMP UI + 모던 문법)

과제4에서 4점 획득한 이유: ShakeCam.cs가 클래스 단위로 완성되어 있어 30% 반영을 받은 것. 다른 코드 우수 학생들도 마찬가지로 “코드는 있되 호출 없음”에 해당한다.

## 7.7 학생15 (35점, D-C 경계) – 코드/실행 극단적 분리 (음영 강조 사례)

### 개요

12개 .cs 파일을 루트 디렉토리(Scripts/ 폴더 없음)에 제출. 시험 요구 5개 체인 전체를 완전 연동한 코드임에도, 자가평가 1/8 (프로젝트만 1) – 게임이 실행조차 안 되었다고 보고.

### 코드 – 표준 답안 그 자체

```

1 public class DamageDealer : MonoBehaviour
2 {
3     [SerializeField] int damage = 1;
4     public int GetDamage() => damage;
5     public void Hit() { Destroy(gameObject); }
6 }

```

Listing 26: 학생15 – DamageDealer.cs

```

1 public class EnemyReward : MonoBehaviour
2 {
3     [SerializeField] int pointValue = 10;
4
5     public void GrantReward()
6     {
7         ScoreKeeper sk = FindAnyObjectByType<ScoreKeeper>();
8         if (sk != null) sk.AddScore(pointValue);
9     }
10 }

```

Listing 27: 학생 15 – EnemyReward.cs (Chain 4 우측)

```

1 public class FriendlyFirePenalty : MonoBehaviour
2 {
3     [SerializeField] int penaltyAmount = 1;
4
5     public void ApplyPenalty()
6     {
7         Submarine sub = FindAnyObjectByType<Submarine>();
8         if (sub != null)
9             sub.GetComponent<Health>().TakeDamage(penaltyAmount);
10    }
11 }

```

Listing 28: 학생 15 – FriendlyFirePenalty.cs (Chain 4 우측)

학생 15의 코드는 **시험의 표준 답안에 가장 근접한 형태**이다. 모든 클래스명, 메서드명, 책임 분리, 신형 Input System 사용, Coroutine 기반 발사 등 시험이 명시한 모든 패턴을 정확히 적용했다.

### 감점 사유 – 35점에 그친 이유

자가평가 1/8은 **Inspector 연결 작업이 시험 시간 내에 거의 완료되지 않았음**을 의미한다. 코드 35점은 다음과 같이 분해된다.

|                              | 점수            |
|------------------------------|---------------|
| 과제1 (자가평가 “입력처리=0” → 30% 반영) | 13/25         |
| 과제2 (자가평가 “적출현=0” → 30% 반영)  | 9/25          |
| 과제3 (자가평가 “게임처리=0” → 30% 반영) | 11/35         |
| 과제4 (ShakeCam 코드 존재 30%)     | 2/15          |
| <b>총점</b>                    | <b>35/100</b> |

이 35점은 “코드 품질에 대한 평가”로는 부당하지만, “시험은 동작 가능 게임을 평가”라는 채점 원칙상 합리적이다. 채점 기록의 회색 음영은 이러한 불일치를 시각적으로 표시한다.

## 7.8 학생 23 (37점, C등급) – 동작하는 게임의 낮은 점수

### 사례 선정 배경

학생 23은 자가평가 7/8로 학생 20과 함께 “실제 플레이 가능”으로 보고된 두 명 중 한 명이다. 그러나 최종 점수는 **37점(C등급)**으로 학생 20(78점)과 41점 격차가 발생하였다.

#### 핵심 진단

학생 23의 코드는 모든 과제에서 동작은 하나, 시험이 평가하고자 한 8개 핵심 아키텍처(SeaWaveConfigSO, Launcher, DamageDealer, Health, LayerMask 분기, GetComponent 분기, ScoreKeeper UI, Wave/Follower 패턴) 중 어느 것도 구현하지 않았다. 9개 .cs 파일 총 약 200줄의 절차적 구현으로 게임은 성립하나, 컴포넌트 기반 설계 평가 항목 다수에서 부분 점수만 인정되었다.

### 과제 1: 잠수함 조종 (11/25)

#### 1-2 조종 스크립트 (7/12) – Submarine.cs

```

1 public class Submarine : MonoBehaviour
2 {
3     public float speed = 5f;
4     public float minY = -4f;
5     public float maxY = 4f;

```



```

6 public GameObject torpedoPrefab;
7 public Transform firePoint;
8
9 void Update() { Move(); Shoot(); }
10
11 void Move()
12 {
13     float input = Input.GetAxis("Vertical");
14     Vector3 pos = transform.position;
15     pos.y += input * speed * Time.deltaTime;
16     pos.y = Mathf.Clamp(pos.y, minY, maxY);
17     transform.position = pos;
18 }
19
20 void Shoot()
21 {
22     if (Input.GetKeyDown(KeyCode.Space))
23         Instantiate(torpedoPrefab, firePoint.position, Quaternion.identity);
24 }
25 }

```

Listing 29: 학생23 – Submarine.cs (전체)

**감점 사유:**

- **구식 Input API 사용:** Input.GetAxis("Vertical"), Input.GetKeyDown(KeyCode.Space) 사용. 시험 요구는 Unity 신형 Input System(InputAction.OnMove 콜백) 기반.
- **필드 노출 방식:** public float 직접 노출 – [SerializeField] private로의 캡슐화 미적용.
- **경계 제한 1-3 (2/6):** 이동 함수 내부에 Mathf.Clamp 한 줄로 처리. 시험 요구는 카메라 ViewportToWorldPoint 기반의 화면 종횡비 자동 대응.

**과제 2: 해류 생물 시스템 (4/25)****2-1 SeaWaveConfigSO (0/6) – ScriptableObject 미존재**

제출된 9개 .cs 파일 어디에도 [CreateAssetMenu] 또는 ScriptableObject 상속 클래스가 없다. 시험에서 가장 핵심적으로 요구한 SeaWaveConfigSO(웨이브 설정 데이터 자산)가 **전무**하다.

**2-3 해류 경로/Wave (0/4) – Friendly.cs**

```

1 public class Friendly : MonoBehaviour
2 {
3     public float speed = 2f;
4     void Update()
5     {
6         transform.Translate(Vector3.left * speed * Time.deltaTime);
7         if (transform.position.x < -10f) Destroy(gameObject);
8     }
9 }

```

Listing 30: 학생23 – Friendly.cs (전체)

해류 경로(웨이포인트, 베지어 곡선, 사인파 등) 개념이 전혀 없음. 단순 직선 이동 1줄로 “해류 생물”이라는 의미를 잃었다. Follower (경로를 추종하는 컴포넌트) 패턴도 부재.

**2-4 Spawner/Follower (2/5) – Spawner.cs**

```

1 void Start() { InvokeRepeating("Spawn", 1f, spawnDelay); }
2 void Spawn()
3 {
4     float y = Random.Range(minY, maxY);
5     Vector3 pos = new Vector3(10f, y, 0);
6     if (Random.value > 0.5f)
7         Instantiate(enemyPrefab, pos, Quaternion.identity);

```

```

8     else
9         Instantiate(friendlyPrefab, pos, Quaternion.identity);
10 }

```

Listing 31: 학생23 – Spawner.cs (핵심)

- **Coroutine 미사용**: InvokeRepeating 문자열 기반 호출.
- **Wave 설정 미연결**: SO 미존재로 인스펙터의 spawnDelay 고정값.
- **Follower 미연결**: SetWaveConfig(...) 같은 데이터 주입 부재.

2-5 난이도 설계 (0/5): Random.value > 0.5f 단순 이진 분기. 시간 경과에 따른 스폰 속도 증가, 비율 변화, 새 적 타입 도입 등 난이도 곡선 설계 전무.

### 과제 3: 전투 시스템 (15/35)

3-1 Launcher (4/9): Launcher 클래스 미존재. Submarine.Shoot() 내부에 Instantiate 한 줄 인라인.

3-2 레이어 설정 (2/5), 3-4 컴포넌트 분기 (4/8) – Torpedo.cs

```

1 void OnTriggerEnter2D(Collider2D other)
2 {
3     if (other.CompareTag("Enemy"))
4     {
5         GameManager.instance.AddScore(Random.Range(10, 31));
6         Destroy(other.gameObject);
7     }
8     else if (other.CompareTag("Friendly"))
9     {
10        GameManager.instance.DamagePlayer(10);
11    }
12    Destroy(gameObject);
13 }

```

Listing 32: 학생23 – Torpedo.cs (충돌 처리)

- **LayerMask 미사용**: CompareTag 문자열 비교.
- **GetComponent 분기 미사용**: 시험 평가 패턴 (var enemy = other.GetComponent<Enemy>()); 부재.
- **보상/벌점 책임 위치**: Torpedo가 직접 처리. 시험 요구는 적/아군 측의 GrantReward()/ApplyPenalty() 메서드.

3-3 Health (2/5), 3-5 DamageDealer (1/3): 두 클래스 모두 미존재. GameManager.playerHP 단일 정수 필드로 모든 체력 처리.

3-6 ScoreKeeper (2/5): Debug.Log("점수: " + score) 콘솔 출력만. UI 표시 부재.

### 과제 4: 연출 효과 (7/15)

4-1 자유 이펙트 (3/9) – Effect.cs

```

1 public class Effect : MonoBehaviour
2 {
3     public float lifeTime = 1f;
4     void Start() { Destroy(gameObject, lifeTime); }
5 }

```

Listing 33: 학생23 – Effect.cs (전체)

단순 "수명 후 파괴" 4줄 클래스. ParticleSystem, Animator 등 실제 시각 효과 로직이 전무.

4-2 수중 충격파 (4/6) – CameraShake.cs: 클래스 자체는 잘 구현되었으나 호출하는 코드가 9개 파일 어디에도 없다. 자가평가 "카메라진동=1"과 코드 실행 사이의 **유일한 모순**.

## 종합: 시험 요구 vs 학생23 구현 매핑

| 시험 요구 아키텍처                         | 학생23 구현 | 대체 방식                               |
|------------------------------------|---------|-------------------------------------|
| SeaWaveConfigSO (ScriptableObject) | 없음      | Spawner.spawnDelay 인스펙터 스칼라         |
| Launcher 클래스                       | 없음      | Submarine.Shoot() 인라인 Instantiate   |
| DamageDealer 클래스                   | 없음      | Torpedo 직접 AddScore/DamagePlayer 호출 |
| Health 클래스                         | 없음      | GameManager.playerHP 단일 필드          |
| LayerMask 기반 충돌 분기                 | 없음      | CompareTag("Enemy"/"Friendly") 문자열  |
| GetComponent<T>() 분기               | 없음      | 위와 동일 (태그 분기)                       |
| ScoreKeeper UI (TextMeshProUGUI)   | 없음      | Debug.Log 콘솔 출력                     |
| Wave/Follower 패턴                   | 없음      | Friendly의 Translate(Vector3.left)   |
| Coroutine 기반 Spawner               | 없음      | InvokeRepeating                     |
| 신형 Input System                    | 없음      | Input.GetAxis/GetKeyDown            |

## 7.9 사례 비교 매트릭스

|                     | 조         | 박정  | 김재  | 옥   | 강   | 박주  | 이   | 최      |
|---------------------|-----------|-----|-----|-----|-----|-----|-----|--------|
|                     | 78        | 52  | 50  | 47  | 46  | 46  | 35  | 37     |
| 표준 클래스명             | 부분        | ◎   | ◎   | ◎   | ◎   | ◎   | ◎   | ×      |
| 신형 InputSystem      | ◎         | ◎   | ◎   | ◎   | ◎   | ◎   | ◎   | ×      |
| SeaWaveConfigSO     | ◎         | ◎   | ◎   | ×   | ◎   | ◎   | ◎   | ×      |
| Coroutine 발사        | ◎         | ◎   | ×   | ◎   | ◎   | ◎   | ◎   | ×      |
| GetComponent 분기     | ◎ (인터페이스) | ◎   | ×   | ◎   | ◎   | ◎   | ◎   | ×      |
| EnemyReward/Penalty | ◎         | 스텝  | ×   | ◎   | ◎   | ◎   | ◎   | ×      |
| TextMeshProUGUI     | ◎         | ×   | ×   | ◎   | ◎   | ◎   | ◎   | ×      |
| CameraShake 호출      | ◎         | ×   | ×   | ×   | ×   | ×   | ×   | ×      |
| 이펙트 자유 구현           | ◎ (VFX)   | ×   | ×   | ×   | ×   | ×   | ×   | × (스텝) |
| 이벤트 시스템             | ◎         | ×   | ×   | ×   | ×   | ×   | ×   | ×      |
| 자가평가                | 7/8       | 4/8 | 3/8 | 4/8 | 2/8 | 2/8 | 1/8 | 7/8    |

관찰: 표의 행을 위에서 아래로 읽으면, 각 행은 하나의 학습 목표이고 ◎ 표시 수가 그 항목 도달자 수이다.

- 쉬운 행(신형 InputSystem, SeaWaveConfigSO, Coroutine, GetComponent 분기): 7-8명 도달.
- 중간 행(EnemyReward/Penalty, TextMeshProUGUI): 5-6명 도달.
- 어려운 행(CameraShake 호출, 이펙트 자유 구현, 이벤트 시스템): 학생20 1명만 도달.

이는 “상위권 안에서도 과제4(연출)와 시스템 통합이 결정적 변별 요인”이라는 점을 명확히 보여준다.

## 8 종합 소견

### 8.1 과제별 완성도 패턴

#### 과제별 완성도 단계

1. **과제 1 (잠수함 조종)** - 평균 17.4/25 (70%): 대부분 완성. Input System + deltaTime 이동은 수업 직후 바로 적용 가능한 수준. 25점 만점이 7명, 0점이 3명(미제출)으로 양극화.
2. **과제 2 (해류 생물)** - 평균 6.4/25 (26%): ScriptableObject와 Spawner 코루틴까지는 약 40%가 도달. 그러나 난이도 설계(2-5)는 학생04·학생22·학생09 3명만 시도. “프로그래머-디자이너 분리”라는 학습 목표가 가장 어려웠다.
3. **과제 3 (전투 시스템)** - 평균 5.0/35 (14%): **가장 큰 격차 발생**. DamageDealer와 Health.OnTriggerEnter2D 연결이 핵심 병목. 25명 중 6명만 완전 연동. 0점자가 30%(8명).
4. **과제 4 (연출 효과)** - 평균 1.5/15 (10%): 극소수만 도달. 카메라 셰이크 “호출 코드” 작성자 4명, 파티클 이펙트 호출 작성자는 **학생20 1명**.

### 8.2 4가지 “불일치 패턴” 종합

본 시험에서 관찰된 학생의 코드 산출물과 자가평가 보고 사이의 불일치는 4가지 유형으로 분류된다.

#### 불일치 패턴 분류

| 유형             | 코드  | 자가평가 | 대표 학생            | 원인                                                |
|----------------|-----|------|------------------|---------------------------------------------------|
| A. 정합(높음)      | 우수  | 우수   | 학생20(78)         | 코드+에디터 모두 완성 - 모범 답안                              |
| B. 정합(낮음)      | 미흡  | 미흡   | 다수(20명)          | 양쪽 모두 시간 부족                                       |
| C. 코드 우수/실행 미완 | 최상  | 1~2  | 학생15, 학생01, 학생09 | Inspector 연결, 프리팹 구성, SO 자산 미작성 - 에디터 워크플로우 학습 부족 |
| D. 동작/표준 미준수   | 비표준 | 우수   | 학생23             | 절차적 구현으로 동작은 가능하나 시험이 평가하는 컴포넌트 패턴 부재 - 설계 학습 부족  |

#### 교육적 의미:

- **유형 C (학생15·학생01·학생09)**는 “프로그래머 능력은 갖췄으나 Unity 워크플로우 능력은 별개”임을 보여준다. 코드 한 줄만 더 쓰면 동작할 게임이 **Inspector 슬롯 하나의 누락**으로 멈춘다.
- **유형 D (학생23)**는 그 반대 방향에서, “디자인 패턴 학습 없이도 단순 게임은 만들 수 있다”는 사실을 보여준다. 그러나 이는 재사용·확장 불가능한 일회용 코드이다.
- **유형 A (학생20)**만이 두 능력의 교집합이며, 그 결과 78점이라는 독보적 점수가 가능했다.

### 8.3 5가지 주요 시사점

#### 1. 개별 스크립트 vs 시스템 연동의 격차

대부분의 학생이 개별 스크립트(Submarine, Launcher, Health) 작성은 가능하나, 스크립트 간 **GetComponent / FindObjectOfType**를 통한 런타임 연동에서 실패한다. 이는 수업에서 “스크립트 간 통신” 주제의 심화 필요성을 시사한다.

#### 2. 코드 작성 능력 ≠ Unity 완성 능력

학생01, 학생09, 학생15은 코드 품질 최상(5개 체인 완전 연동)이나 자가평가에서 대부분 미동작 보고. 이는 **에디터 워크플로우** (프리팹 구성, Inspector 할당, SO 에셋 생성, 레이어 설정) 능력이 코드 작성 능력과 별개임을 보여준다. 향후 수업에서는 **매 코드 작성 실습마다 “에디터 연결”까지 한 사이클로 완료**하도록 지도 방식을 조정할 필요가 있다.

#### 3. 표준 패턴 vs 절차적 우회

학생23의 사례는 “되는 게임”이 “평가받는 게임”과 다를 수 있음을 보여준다. AI 도구 허용 환경에서, 학생은 (a) 시험이

요구하는 표준 패턴을 정확히 인지·구현하거나 (b) 자기가 익숙한 단순 구조로 우회할 수 있다. 시험 안내 시 **“명시된 클래스명·패턴이 채점 대상”임을 명문화**해야 한다.

#### 4. AI 도구 활용의 양면성

AI 도구 허용 환경에서 코드 품질은 전반적으로 향상되었으나(과제3 Chain 3 완전 연동 6명은 작년 대비 2배), **생성된 코드를 Unity에서 동작시키는 통합 능력**은 여전히 학생 본인의 역량에 의존한다. 학생20과 학생23만이 실제 플레이 가능한 수준으로 완성하였다는 사실은 AI 도구가 설계와 통합 능력의 부족을 메워주지는 못한다는 분명한 증거이다.

#### 5. 과제4(연출)의 극단적 미달성

74%가 과제4에서 0점이고, 카메라 셰이크 호출 코드 작성자는 4명, 파티클 이펙트 호출자는 1명에 불과하다. 이는 **시험 시간 분배 실패**의 신호이다 (학생들이 과제3에서 막히면서 과제4까지 도달하지 못함). 차기 시험에서는 **과제별 시간 권장량 명시** 또는 **과제 의존성 완화**(과제4를 과제3과 분리 가능하게)를 검토해야 한다.

## 8.4 향후 수업 방향

### 다음 학기 개선 항목

- **“프리랩 구성 실습” 시간 확대**  
매 강의에서 마지막 15분을 Inspector 연결에 할당. 코드 작성 후 “드래그 앤 드롭으로 게임을 동작시키기”까지 한 사이클 완료.
- **과제 3 “컴포넌트 분기” 패턴 단계별 실습**  
GetComponent<T>(), TryGetComponent<T>(), IDamageable 인터페이스 패턴을 **3주에 걸쳐 단계적**으로 소개. 작년 기말은 1주에 압축되어 있었다.
- **기말고사 형식 검토**  
기본 프로젝트(잠수함과 적이 이미 동작하는 씬) 제공 후 **기능 확장** 방식으로 변경 검토. “처음부터 만들기”는 시험 시간 내 시스템 통합을 어렵게 만든다.
- **표준 패턴의 사전 명시**  
중간고사 안내문에 “채점 대상 클래스명/메서드명 목록”을 사전 공지. 학생23 유형의 “우회 구현”으로 인한 점수 손실 방지.
- **자가평가 양식 개선**  
8개 항목별 “코드 작성 여부”와 “Inspector 연결 여부”를 분리 보고 하도록 양식을 분리. 유형 C(코드/실행 불일치)의 진단 정밀도 향상.